API AUTOMATION FRAMEWORK REPORT

Petstore API Testing Project

Prepared by:
Rajashree Satapathy

1. Introduction

This project is about building a **solid, hybrid framework** to automate testing for the Petstore's **RESTful APIs**. It is designed with scalability and long-term maintenance in mind, using a modern tech stack that aligns with how the industry actually works today.

To keep things clear for everyone—engineers and stakeholders alike—**BDD (Behavior Driven Development)** approach with Cucumber is used. This means the test scenarios are written in plain English, making the logic easy to follow without needing to dive deep into the code.

Under the hood, the framework is powered by **REST Assured** for the heavy lifting and **Maven** for smooth build management. It could also handle more complex, real-world tasks like dynamic data injection, API chaining, and generating detailed reports. Ultimately, this project is a practical look at how to tackle common automation hurdles, like managing flaky responses and building a tool that actually lasts.

2. Objectives

The primary objectives of this project are:

- To develop an API automation framework using **Java and REST Assured**
- To implement **BDD using Cucumber**
- To support **data-driven and dynamic testing**
- To enable execution through **Maven commands**
- To generate **test execution reports**
- To simulate real-world automation framework architecture

3. Technology Stack

Technology	Purpose
Java	Core programming language
REST Assured	API automation library
Cucumber	BDD framework
Maven	Build and dependency management
TestNG	Test execution framework
IntelliJ IDEA	Integrated Development Environment
Git & GitHub	Version control and repository hosting
Postman	API validation and testing
Log4j	Logging and debugging

4. GitHub Repository

The complete project source code is available at:

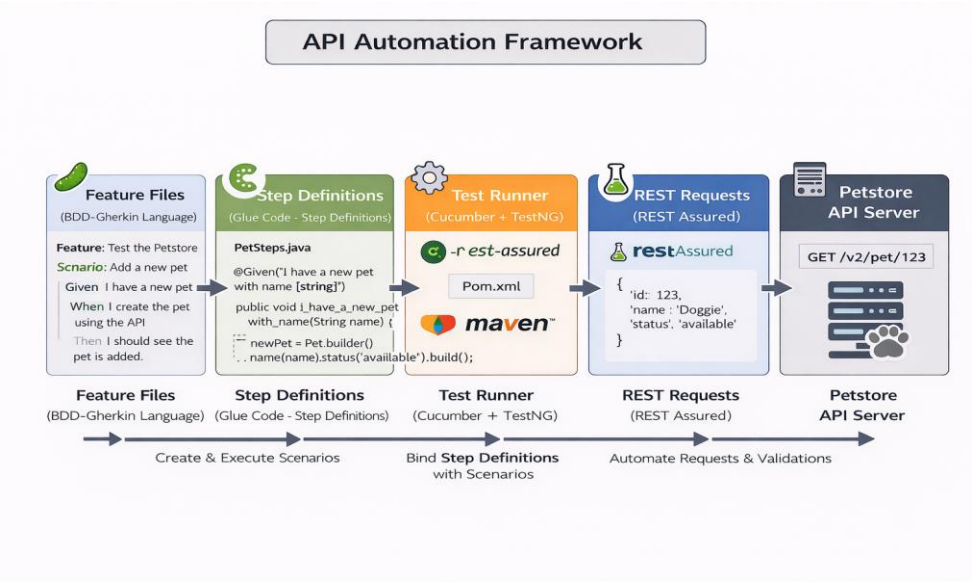
GitHub Repository:
(URL: https://github.com/rajshree601/API-Test-Automation)

The repository includes:

- Full framework implementation
- Feature files and step definitions
- Maven configuration
- Postman collection
- Documentation

5. Project Architecture

The framework follows a **layered and modular architecture** to ensure separation of concerns and reusability.




```
src
└── test
    ├── java
    │   ├── base
    │   ├── hooks
    │   ├── runners
    │   ├── stepDefinitions
    │   ├── utils
    │   └── pojo
    └── resources
        ├── features
        ├── config
        └── log4j.properties
```

Architecture Explanation

- **Base Layer (base)**
  Handles initialization such as base URI setup for API calls.
- **Step Definitions (stepDefinitions)**
  Contains implementation of BDD steps mapped from feature files.
- **Feature Files (features)**
  Written in Gherkin syntax to define test scenarios in a readable format.
- **Runner (runners)**
  Executes feature files using Cucumber and integrates with TestNG.
- **Utilities (utils)**
  Provides reusable methods for request creation and response handling.
- **POJO Classes (pojo)**
  Used for mapping JSON request/response bodies to Java objects.
- **Hooks (hooks)**
  Handles setup and teardown operations before and after test execution.
- **Logging (log4j.properties)**
  Enables logging of execution steps for debugging.

6. Test Scenarios Implemented

6.1 Pet Lifecycle (CRUD Operations)

- Create a new pet using POST request
- Extract pet ID from response
- Retrieve pet details using GET request
- Update pet status using PUT request
- Delete the pet using DELETE request

Validation:

- Pet details match input data
- Deletion is confirmed through status codes

6.2 Inventory Analysis

- Retrieve inventory data using /store/inventory
- Parse response to identify available pets
- Compare with results from /pet/findByStatus

Validation:

- Counts from both endpoints match

6.3 User Security and Negative Testing

- Create user with invalid email format
- Attempt to retrieve a non-existing user
- Attempt login with invalid credentials

Validation:

- Proper error handling (404 for missing user)
- Login validation handled considering API limitations

6.4 Cross Endpoint Data Consistency

- Create a pet with specific attributes
- Update pet status
- Fetch list of pets by status

Validation:

- Confirm that created pet exists in updated dataset

7. Data Handling Strategy

Dynamic test data is generated using:

System.currentTimeMillis()

This ensures:

- Unique data for each execution
- Avoidance of duplication
- Reliable chaining across API calls

8. Postman Implementation

The project includes a Postman collection demonstrating:

- GET requests with query parameters
- POST requests with JSON payload
- Status code validation (200, 404, 400/405)
- Use of environment variables ({{url}})

9. Reporting

Test execution reports are generated using Cucumber and stored at:

target/cucumber-report.html

The report provides:

- Scenario execution status
- Step-by-step results
- Execution duration

10. Maven Execution

The framework supports execution via:

mvn clean test

This performs:

- Project compilation
- Test execution
- Report generation

11. Challenges and Solutions

API Inconsistency

The Petstore API exhibits inconsistent behavior, particularly in authentication responses.

> **Solution:**
> Flexible assertions were implemented to ensure stability without compromising validation.

Data Persistence Issues

Occasional invalid or null responses were observed.

> **Solution:**
> Proper request structure and dynamic IDs were used to ensure data integrity.

Debugging Complexity

Identifying issues during execution required detailed logging.

> **Solution:**
> Logging and response outputs were incorporated for easier debugging.

12. Key Features

- BDD-based automation using Cucumber
- Modular and scalable framework design
- Reusable utility classes
- Dynamic test data generation
- API chaining implementation
- Maven-based execution
- Integrated reporting

13. Conclusion

The project demonstrates the development of a **reliable and scalable API automation framework**. It reflects a clear understanding of automation tools, framework design, and how to handle real-world testing challenges such as dynamic data and API inconsistencies.

The framework is designed to be flexible and maintainable, making it easy to extend with additional features. It can be further enhanced and adapted for use in larger, enterprise-level applications.

14. Future Enhancements

- Integration with Extent Reports
- Continuous Integration using Jenkins
- Data-driven testing using external sources
- Docker-based execution environment
- Enhanced logging and monitoring